Демидовский университет

Графики в Qt

Автор: доцент, кандидат технических наук
Ксения Владимировна Лагутина

Pandas: DataFrame

- DataFrame – таблица, колонки – Series
- Методы DataFrame и Series часто аналогичны

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html

```
df = pd.DataFrame( {
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)),
  dtype="float32"),
        "D": [3] * 4,
        "E": pd.Categorical(["test", "train", "test",
  "train"]),
        "F": "foo",
    }
  )
  print(df)
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

Демидовский университет

Атрибуты (поля, свойства) DataFrame

- dtypes – типы данных колонок
- index – индексы строк
- columns – названия колонок
- at[i, j], iat[i, j] – ячейка по двум индексам/ключам
- iloc[key], loc[key] – строка или столбец по индексу/ключу
- size – количество ячеек
- shape – двумерный размер
- T – транспонированная таблица

Демидовский университет

Методы DataFrame

- head() - начало таблицы
- tail(n) – последние n строк
- describe() - описание таблицы
- sort_index(axis=1, ascending=False) – сортировка колонок по названию
- sort_values(by="B") – сортировка строк по колонке "B"
- mean(), median(), max(), min() - статистика по колонкам
- apply(func) – применить функцию func к данным

Демидовский университет

Методы DataFrame

- groupby(["A", "B"]).sum() - группировка по колонкам с суммированием
- read_csv("foo.csv") – чтение из csv-файла
- to_csv("foo.csv") – запись в csv-файл
- pd.concat([df1, df2]) – объединить таблицы с одинаковыми столбцами

Демидовский университет

Формат CSV

- Comma-Separated Values — значения, разделённые запятыми
- Текстовый формат, предназначенный для представления табличных данных

Last name,First name,SSN,Test1,Test2,Test3,Test4,Final,Grade

Alfalfa,Aloysius,123-45-6789,40.0,90.0,100.0,83.0,49.0,D-

Alfred,University,123-12-1234,41.0,97.0,96.0,97.0,48.0,D+

Демидовский университет

Графики: Qt Charts

- Qt Charts предоставляет набор компонентов диаграмм
- Использует платформу Qt Graphics View Framework для интеграции диаграмм с пользовательскими интерфейсами
- Диаграммы могут использоваться как типы QWidgets, QGraphicsWidget или QML

https://doc.qt.io/qtforpython/PySide6/QtCharts/index.html

https://doc.qt.io/qtforpython/examples/index.html#charts

Демидовский университет

QChart – основа графика

- QChart – это QGraphicsWidget , который можно отобразить в QGraphicScene
- Управляет графическим представлением различных типов рядов и других объектов, связанных с диаграммами, таких как условные обозначения и оси
- Чтобы отобразить диаграмму в интерфейсе, следует использовать QChartView
- Полярные диаграммы отображаются с помощью класса QPolarChart

https://doc.qt.io/qtforpython/PySide6/QtCharts/QChart.html

Демидовский университет

QChart – основа графика

- legend() – объект легенды
- addSeries() – добавить данные
- addAxis(), axes() – управление осями
- createDefaultAxes() – добавить оси к имеющимся данным
- setTitle(title) – заголовок
- zoom(factor) – масштабирование
- QChartView отображает график
  - setChart(chart) – указать экземпляр QChart

Демидовский университет

Серии данных

- QAreaSeries – двумерная фигура
- QBarSeries – столбчатая диаграмма
- QBoxPlotSeries – коробчатая диаграмма
- QCandlestickSeries – диаграмма-свечка
- QLineSeries – линейный график
- QPercentBarSeries – столбчатая диаграмма процентов
- QPieSeries – круговая диаграмма
- QScatterSeries – точечная диаграмма
- QSplineSeries – сплайн
- QStackedBarSeries – столбчатая диаграмма с категоризацией
- QHorizontal*Series – горизонтальные версии диаграмм

https://doc.qt.io/qtforpython-6/PySide6/QtCharts/index.html#module-PySide6.QtCharts

Демидовский университет

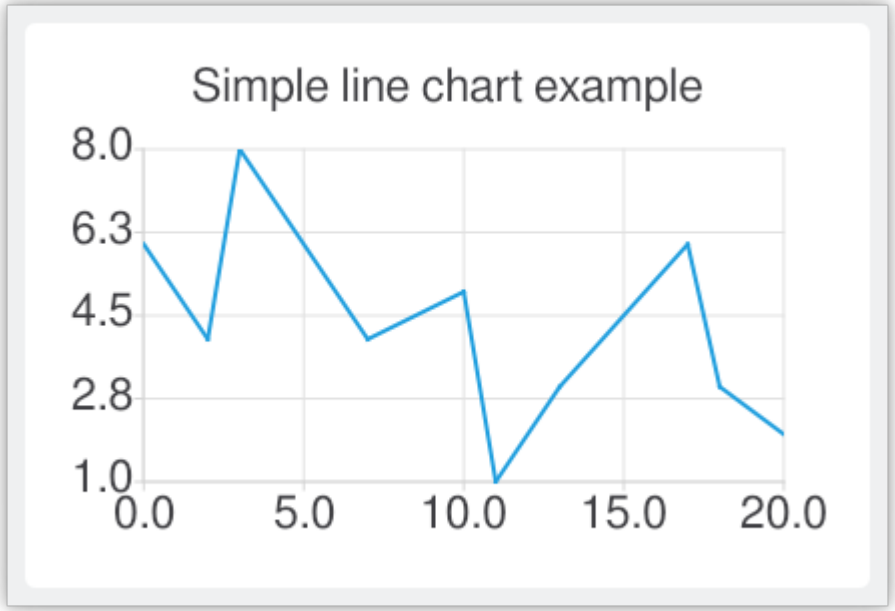
Базовые классы серий

- QAbstractSeries – общий базовый класс
  - attachAxis(axis) – добавить ось
- QXYSeries – базовый класс для графиков на декартовой плоскости
  - append() – добавить данные
- QAbstractBarSeries – базовый класс для столбчатых диаграмм
  - append() – добавить данные

Демидовский университет

Пример линейного графика

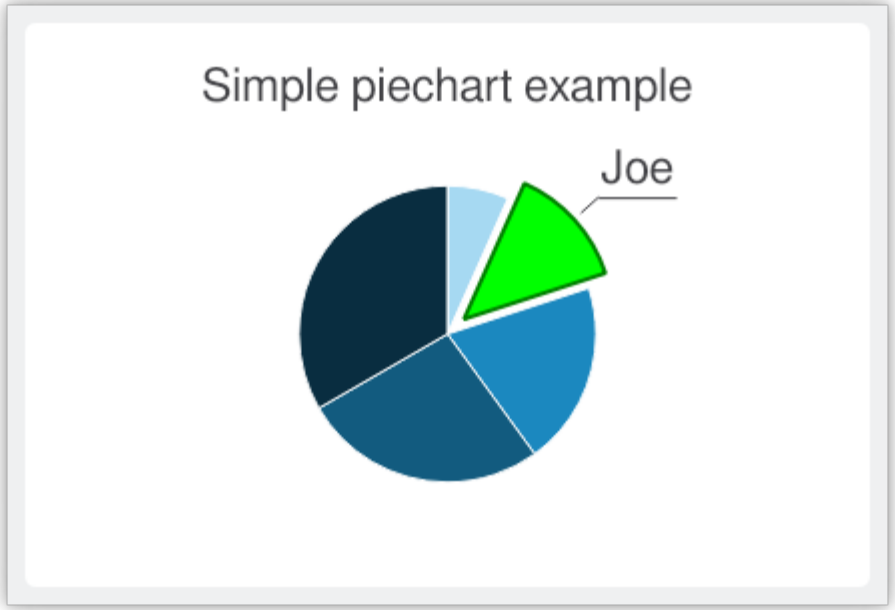
```
class TestChart(QMainWindow):
    def __init__(self):
        super().__init__()
        self.series = QLineSeries()
        self.series.append(0, 6)
        self.series.append(2, 4)
        self.series.append(3, 8)
        self.series.append(7, 4)
        self.series.append(10, 5)
        self.series.append(QPointF(11, 1))
        self.series.append(QPointF(13, 3))
        self.series.append(QPointF(17, 6))
        self.series.append(QPointF(18, 3))
        self.series.append(QPointF(20, 2))
        self.chart = QChart()
        self.chart.legend().hide()
        self.chart.addSeries(self.series)
        self.chart.createDefaultAxes()
        self.chart.setTitle("Simple line chart example")
        self._chart_view = QChartView(self.chart)
        self._chart_view.setRenderHint(QPainter.Antialiasing)
        self.setCentralWidget(self._chart_view)
```



Демидовский университет

Пример круговой диаграммы

```
class TestChart(QMainWindow):
    def __init__(self):
        super().__init__()
        self.series = QPieSeries()
        self.series.append('Jane', 1)
        self.series.append('Joe', 2)
        self.series.append('Andy', 3)
        self.series.append('Barbara', 4)
        self.series.append('Axel', 5)
        self.slice = self.series.slices()[1]
        self.slice.setExploded()
        self.slice.setLabelVisible()
        self.slice.setPen(QPen(Qt.darkGreen, 2))
        self.slice.setBrush(Qt.green)
        self.chart = QChart()
        self.chart.addSeries(self.series)
        self.chart.setTitle('Simple piechart example')
        self.chart.legend().hide()
        self._chart_view = QChartView(self.chart)
        self._chart_view.setRenderHint(QPainter.Antialiasing)
        self.setCentralWidget(self._chart_view)
```



Демидовский университет

Наборы категориальных даных

- QBarSet – для столбчатой диаграммы
- QBoxSet – для коробчатой диаграммы
- QCandlestickSet – для диаграммы-свечки
- append() – добавить данные
- Имя категории можно задать в конструкторе

Демидовский университет

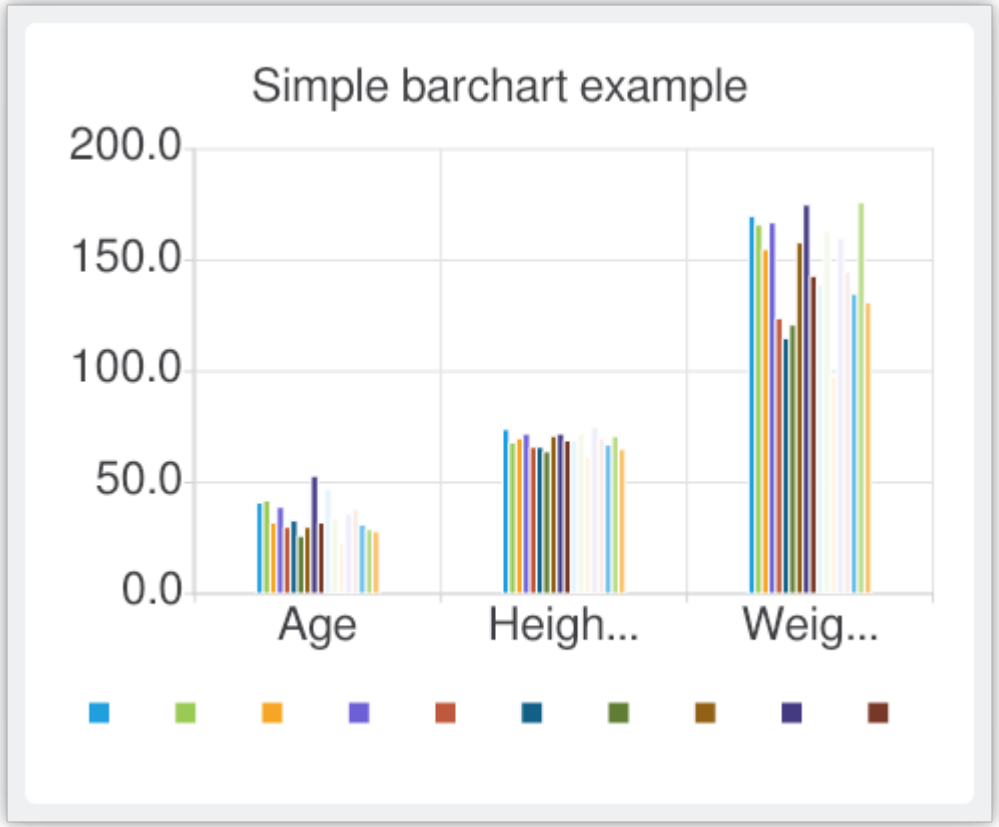
Оси

- QAbstractAxis – базовый класс для осей
- QValueAxis – ось значений с метками, линиями сетки и оттенками
  - setRange() – диапазон значений
  - setTickCount() – количество меток
  - setLabelFormat() – формат текста
- QLogValueAxis – логарифмическая шкала
- QDateTimeAxis – дата и время
- QCategoryAxis и QBarCategoryAxis – категориальные оси

Демидовский университет

Пример столбчатой диаграммы

```
df = pd.read_csv("biostats.csv")
series = QBarSeries()
for i in range(len(df.index)):
    set = QBarSet(df.iloc[i]['Name'])
    set.append(df.iloc[i, 2:].values.flatten().tolist())
    series.append(set)
self.chart = QChart()
self.chart.addSeries(series)
self.categories = df.columns[2:]
self.axis_x = QBarCategoryAxis()
self.axis_x.append(self.categories)
self.chart.addAxis(self.axis_x, Qt.AlignBottom)
series.attachAxis(self.axis_x)
self.axis_y = QValueAxis()
self.axis_y.setRange(0, 200)
self.chart.addAxis(self.axis_y, Qt.AlignLeft)
series.attachAxis(self.axis_y)
self.chart.legend().setVisible(True)
self.chart.legend().setAlignment(Qt.AlignBottom)
self._chart_view = QChartView(self.chart)
```



Демидовский университет

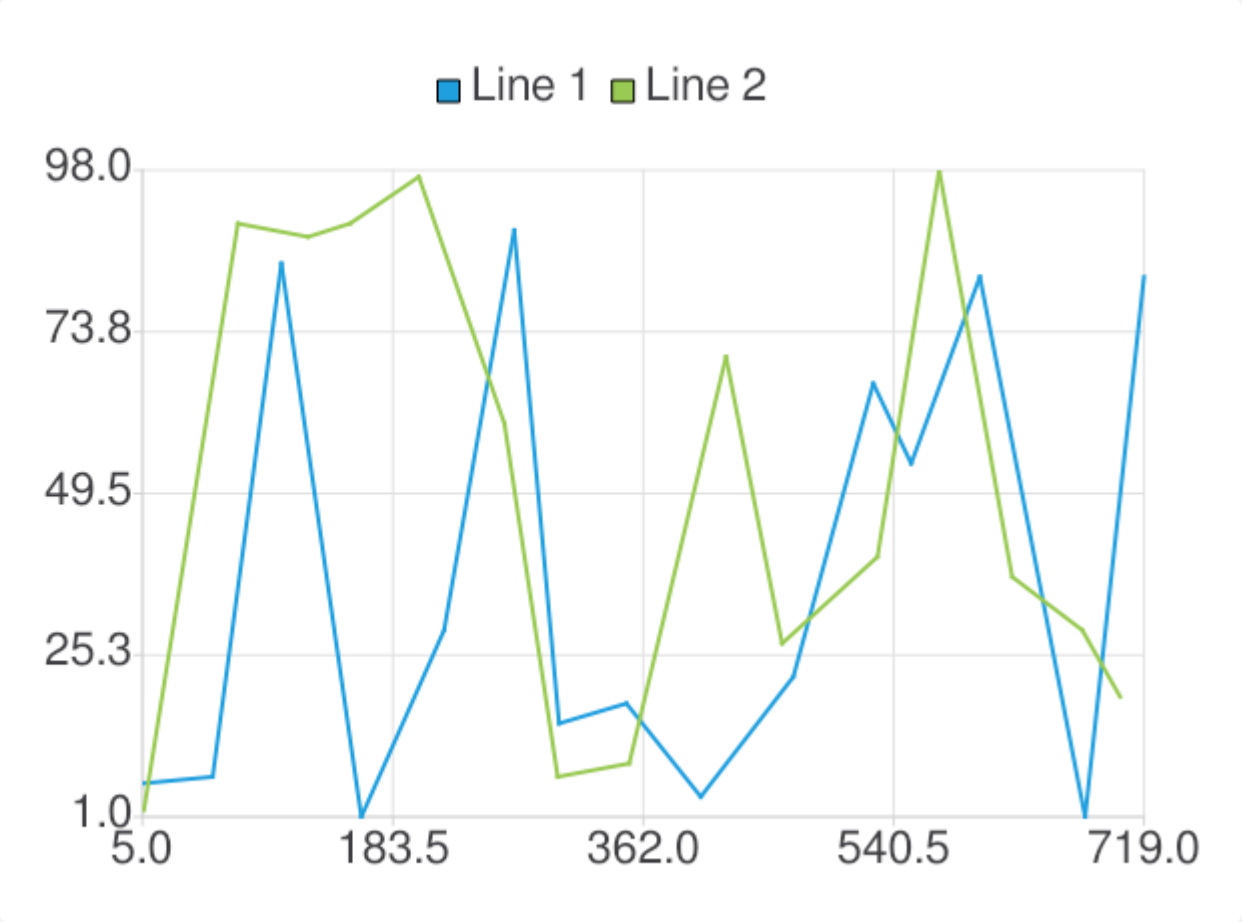
Данные из модели

- Q*Mapper – классы для сопоставления
- QVXYModelMapper – вертикальное сопоставление для координатной плоскости
  - setXColumn(), setYColumn() – сопоставление оси с колонкой таблицы
  - setSeries() – сопоставление с серией (серия без данных)
  - setModel() – сопоставление с моделью

Демидовский университет

Пример данных из модели

	x	y	x	y
1	5	6	6	2
2	55	7	73	90
3	104	84	123	88
4	161	1	153	90
5	220	29	202	97
6	270	89	263	60
7	302	15	301	7
8	350	18	352	9
9	403	4	421	70
10	469	22	461	27
11	526	66	529	40
12	553	54	573	98
13	602	82	625	37
14	677	1	675	29
15	719	82	702	19



Демидовский университет

Пример данных из модели

```
class Window(QWidget):
    def __init__(self):
        super().__init__()
        self.model = CustomTableModel()
        self.table_view = QTableView()
        self.table_view.setModel(self.model)
        self.chart = QChart()
        self.chart.setAnimationOptions(QChart.AllAnimations)
        self.series = QLineSeries()
        self.series.setName("Line 1")
        self.mapper = QVXYModelMapper(self)
        self.mapper.setXColumn(0)
        self.mapper.setYColumn(1)
        self.mapper.setSeries(self.series)
        self.mapper.setModel(self.model)
        self.chart.addSeries(self.series)
        self.model.add_mapping(self.series.pen().color().name(), QRect(0, 0, 2, self.model.rowCount()))
        self.series = QLineSeries()
        self.series.setName("Line 2")
        self.mapper = QVXYModelMapper(self)
        self.mapper.setXColumn(2)
        self.mapper.setYColumn(3)
        self.mapper.setSeries(self.series)
        self.mapper.setModel(self.model)
        self.chart.addSeries(self.series)
        self.model.add_mapping(self.series.pen().color().name(), QRect(2, 0, 2, self.model.rowCount()))
        self.chart.createDefaultAxes()
        self.chart_view = QChartView(self.chart)
        self.chart_view.setRenderHint(QPainter.Antialiasing)
        self.chart_view.setMinimumSize(640, 480)
```

Демидовский университет

Библиотека PyQtGraph

- PyQtGraph – это библиотека для графики, написанная на чистом Python, построенная на PyQt / PySide и numpy
- Предназначена для использования в математических/научных/инженерных приложениях
- Работает быстро благодаря использованию NumPy для обработки чисел и фреймворка Qt GraphicsView для отображения
- PyQtGraph распространяется по лицензии MIT с открытым исходным кодом

https://www.pyqtgraph.org/

Демидовский университет

Возможности библиотеки

- Линейные и точечные 2D-графики
  - Данные можно перемещать / масштабировать с помощью мыши
- Отображение изображений с интерактивными таблицами поиска и контролем уровня
- Система 3D-графики (требуется библиотека Python+OpenGL, например PyOpenGL)
- Отбор/маркировка данных и контроль интересующей области
  - Виджеты для выбора произвольных областей на изображениях и автоматической нарезки данных в соответствии с ними
- Библиотека виджетов и модулей, полезных для научных / инженерных приложений
  - Блок-схемы для интерактивного прототипирования.
  - Виджет для отображения/редактирования иерархий объектов
- …

Демидовский университет

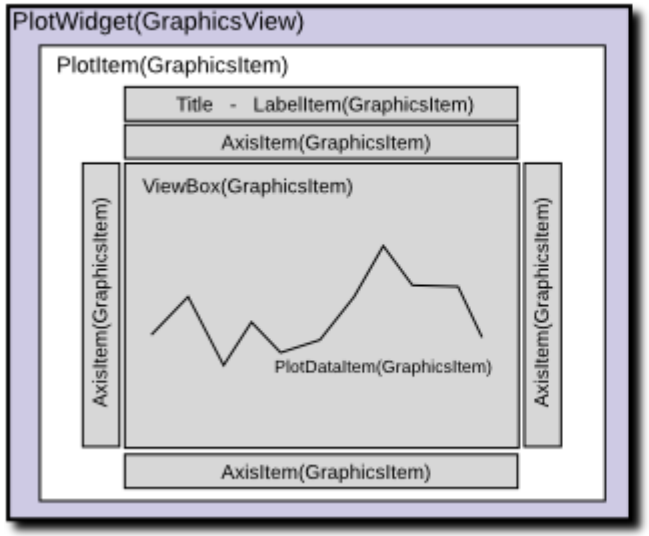
Базовые функции построения графиков

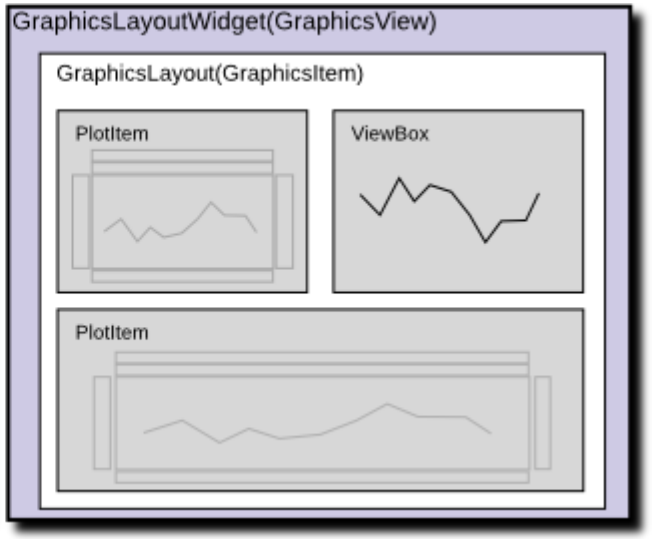
- pyqtgraph.plot() – создает новое окно графика, отображающее данные
- PlotItem.plot() – добавляет новый набор данных в существующий виджет графика
  - PlotWidget.plot() вызывает PlotItem.plot
- GraphicsLayout.addPlot() – добавляет новый график в сетку
- Аргументы:
  - x – опциональные целочисленные данные по оси X; если не указаны, то диапазон целых чисел будет сгенерирован автоматически
  - y – данные по оси Y
  - pen – перо для рисования линий графика
  - symbol – строка, описывающая форму символов, которые будут использоваться для каждой точки
  - symbolPen - перо для рисования контура символов
  - symbolBrush – перо для заполнения символов
  - fillLevel – заполняет область под кривой графика заданным Y
  - brush – кисть, используемая при заполнении области под кривой

Демидовский университет

Основные классы

- Классы данных (подклассы QGraphicsItem)
  - PlotCurveItem – график-линия
  - ScatterPlotItem – график из точек
  - PlotDataItem – объединяет предыдущие
- Классы контейнеров (подклассы QGraphicsItem)
  - PlotItem – содержит канву для графиков, оси, метки, заголовок
  - GraphicsLayout – отображает сетку элементов; для нескольких графиков
  - ViewBox – класс для отображения интерактивных данных
  - AxisItem – отображает значения оси, метки и подписи
- Классы контейнеров (подклассы QWidget)
  - PlotWidget - класс с отображаемым единственным графиком
  - GraphicsLayoutWidget - класс, отображающий один GraphicsLayout





Демидовский университет

PlotWidget, PlotItem — основа графика

- addItem, removeItem, clear – управление графиками на канве
- setAxisItems – управление осями
- setXRange, setYRange, setRange – управление размером
- setMouseEnabled – интерактивный ли график
- setBackground – фон графика
- setTitle – заголовок графика
- setLabel – подписи к осям и настройка их стиля
- addLegend – легенда
- showGrid – сетка на канве

Демидовский университет

https://pyqtgraph.readthedocs.io/en/latest/api_reference/widgets/plotwidget.html

mkPen — кисть для линии графика

- Создает экземпляр QPen
- mkPen(color) # Можно задать объект QColor
- mkPen(color, width=2)
- mkPen(cosmetic=False, width=4.5, color='r')
- mkPen({'color': "#FF0", width: 2})

https://pyqtgraph.readthedocs.io/en/latest/api_reference/functions.html#pyqtgraph.mkPen

https://doc.qt.io/qtforpython/PySide6/QtGui/QPen.html

Демидовский университет

Пример встраивания графика в Qt-приложение

```
class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.graphWidget = pg.PlotWidget()
        self.setCentralWidget(self.graphWidget)
        hour = [1,2,3,4,5,6,7,8,9,10]
        temperature = [30,32,34,32,33,31,29,32,35,45]
        self.graphWidget.setBackground('w')
        self.graphWidget.setTitle("Your Title Here", color="b", size="30pt")
        styles = {"color": "#f00", "font-size": "20px"}
        self.graphWidget.setLabel("left", "Temperature (°C)", **styles)
        self.graphWidget.setLabel("bottom", "Hour (H)", **styles)
        self.graphWidget.addLegend()
        self.graphWidget.showGrid(x=True, y=True)
        self.graphWidget.setXRange(0, 10, padding=0)
        self.graphWidget.setYRange(20, 55, padding=0)
        pen = pg.mkPen(color=(255, 0, 0), width=8, style=Qt.SolidLine)
        self.graphWidget.plot(hour, temperature, name="Sensor 1",  pen=pen,
 symbol='+', symbolSize=30, symbolBrush=('b'))
```

Демидовский университет

Вспомогательные функции

- image – создать ImageView для отображения картинок
- dbg – создать консольное окно для отслеживания лога
- mkColor – создать экземпляр QColor
- mkBrush – создать кисть для рисования
- affineSlice – срез данных

https://pyqtgraph.readthedocs.io/en/latest/api_reference/functions.html

Демидовский университет

Классы данных и объектов на графике

- BarGraphItem – столбчатая диаграмма
- IsocurveItem – сплайн
- TextItem – текст на графике
- ArrowItem – стрелка
- FillBetweenItem – заполняет пространство между графиками
- LegendItem – легенда
- DateAxisItem – ось с датой и временем
- MultiPlotItem – несколько графиков на соседних полях

Демидовский университет

Пример столбчатой диаграммы

```
class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        plot = pg.plot()
        df = pd.read_csv("biostats.csv")
        y1 = df.iloc[:,2].values.flatten().tolist()
        x = [i+1 for i in range(len(y1))]
        bargraph = pg.BarGraphItem(x=x, height=y1, width=0.6, brush ='g')
        plot.addItem(bargraph)
        self.setCentralWidget(plot)
        ax = plot.getAxis('bottom')
        xlab = df.iloc[:,0].values.flatten().tolist()
        ticks = [list(zip(x, xlab))]
        ax.setTicks(ticks)
```

кий
ет

QTimer – таймер

- Повторяющийся или одноразовый таймер
- setInterval(msec) – интервал в миллисекундах
- start() – запустить таймер
- timeout() – сигнал об истечении таймера
- setSingleShot() – повторяющийся или одноразовый
- setTimerType() – точность таймера

https://doc.qt.io/qtforpython/PySide6/QtCore/QTimer.html

Демидовский университет

Пример динамического изменения

```
class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.graphWidget = pg.PlotWidget()
        self.setCentralWidget(self.graphWidget)
        self.x = list(range(100))
        self.y = [randint(0,100) for _ in range(100)]
        self.graphWidget.setBackground('w')
        pen = pg.mkPen(color=(255, 0, 0))
        self.data_line = self.graphWidget.plot(self.x, self.y, pen=pen)
        self.timer = QTimer()
        self.timer.setInterval(50)
        self.timer.timeout.connect(self.update_plot_data)
        self.timer.start()
    def update_plot_data(self):
        self.x = self.x[1:]
        self.x.append(self.x[-1] + 1)
        self.y = self.y[1:]
        self.y.append(randint(0, 100))
        self.data_line.setData(self.x, self.y)
```

Демидовский университет